

BIS-SATHI — FINAL API, JSON & AI SERVICE GUIDE

Node/Express REST APIs + FastAPI AI Service — complete connection and JSON contract

Source rule: The uploaded blueprint defines 24 public APIs and identifies **POST /api/v1/chat** as the AI-enabled public API. It names the internal FastAPI chat endpoint as **POST /chat**. This guide keeps that contract and does not invent extra public AI endpoints.

1. FINAL API SPLIT

Layer	API	Who calls it?	What comes from it?
Node/Express Public REST	All /api/v1/* public routes	React / Mobile	Auth, sessions, standards, QCOs, labs, reports, resources, compliance, comparison, saved data and public chat entry
FastAPI Private AI	POST /chat	Node only	AI answer, intent, status, citations and context

2. FASTAPI AI SERVICE — EXACT API

POST /chat is the main internal AI API. It is not a separate API for every AI step. Intent detection, RAG, Pinecone retrieval, LLM generation and citation handling happen inside this service while processing /chat.

FastAPI Request JSON

```
{
  "sessionId": "s123",
  "message": "QCO for Toys",
  "language": "en",
  "context": {
    "standardId": "IS-1293-2019"
  }
}
```

FastAPI Success Response JSON

```
{
  "requestId": "r-123",
  "status": "success",
  "intent": "standard_research",
  "answer": "Grounded answer...",
  "citations": [],
  "context": {}
}
```

FastAPI Insufficient Evidence Response

```
{
  "requestId": "r-124",
  "status": "insufficient_evidence",
  "intent": "standard_research",
  "answer": "",
  "citations": [],
  "context": {}
}
```

Allowed chat status values: **success**, **insufficient_evidence**, **error**, **processing**.

3. WHAT ALL WILL COME FROM FASTAPI?

Field	Meaning
requestId	Request tracking ID
status	AI processing/result status
intent	Detected user-query intent
answer	Final grounded AI answer
citations[]	Evidence/source details used for the answer
context	Relevant response context

Important: AI Sathi ka final AI response FastAPI /chat se aayega. Normal application data is still served by Node REST APIs.

4. NODE → FASTAPI CONNECTION — STEP BY STEP

```
1. React sends:
   POST /api/v1/chat
```



So Node aur FastAPI code ko merge nahi karna hai. Node simply FastAPI ke HTTP endpoint ko call karega aur JSON response receive karega.

5. EXACT NODE → FASTAPI EXAMPLE

Frontend → Node public request

```
{
  "method": "POST",
  "endpoint": "/api/v1/chat",
  "body": {
    "sessionId": "s123",
    "message": "QCO for Toys",
    "language": "en"
  }
}
```

Node → FastAPI internal request

```
{
  "method": "POST",
  "endpoint": "/chat",
  "body": {
    "sessionId": "s123",
    "message": "QCO for Toys",
    "language": "en"
  }
}
```

FastAPI → Node response

```
{
  "requestId": "r-123",
  "status": "success",
  "intent": "standard_research",
  "answer": "Grounded answer...",
  "citations": [],
  "context": {}
}
```

6. AI PROCESSING STEPS — API NAHI, /chat KE ANDAR

Step	Work	Separate endpoint?
Language/query processing	Input clean/normalize	No — inside /chat
Intent detection	Question ka intent identify	No — inside /chat
RAG retrieval	Relevant BIS evidence retrieve	No — inside /chat
Pinecone search	Relevant vector chunks/evidence	No public API
LLM	Evidence-based answer generate	No public API
Citation validation	Evidence/citation check	No — inside /chat

Developer internally separate Python functions/modules bana sakta hai, but frontend contract ke liye ye sab ek AI service ke /chat request ka part hai.

7. COMPLETE 24 PUBLIC NODE/EXPRESS REST APIs

Method	Endpoint	Purpose	AI?
POST	/api/v1/auth/login	Login	No
POST	/api/v1/auth/register	Register	No
POST	/api/v1/auth/logout	Logout	No
GET	/api/v1/auth/me	Profile	No
POST	/api/v1/chat	AI Q&A; → FastAPI	Yes
POST	/api/v1/sessions	Create session	No
GET	/api/v1/sessions	Chat history	No
GET	/api/v1/sessions/:id	Session detail	No
GET	/api/v1/sessions/:id/messages	Messages	No
GET	/api/v1/standards	Standards list/search	No
GET	/api/v1/standards/:id	Standard detail	No
GET	/api/v1/qcos	QCO list	No
GET	/api/v1/qcos/:id	QCO detail	No
GET	/api/v1/labs	Find labs	No
GET	/api/v1/labs/:id	Lab detail	No
GET	/api/v1/reports	Reports list	No
GET	/api/v1/reports/:id	Report detail	No
GET	/api/v1/resources	Resources list	No
GET	/api/v1/resources/:id	Resource detail	No
GET	/api/v1/compliance/journeys	Compliance journeys	No
POST	/api/v1/comparison	Compare entities	No
GET	/api/v1/saved	Saved items	No
POST	/api/v1/saved	Save item	No
DELETE	/api/v1/saved/:id	Remove saved item	No

8. NODE AUTH APIs — JSON

POST /api/v1/auth/login

Purpose: Login

Request JSON:

```
{
  "email": "user@test.com",
  "password": "123"
}
```

Response JSON:

```
{
  "success": true,
  "data": {
    "token": "jwt...",
    "user": {}
  }
}
```

POST /api/v1/auth/register

Purpose: Register

Request JSON:

```
{
  "name": "User",
  "email": "user@test.com",
  "password": "123"
}
```

Response JSON:

```
{
  "success": true,
  "data": {
    "token": "jwt...",
  }
}
```

```
    "user": {}
  }
}
```

POST /api/v1/auth/logout

Purpose: Logout

Request JSON:

```
{}
```

Response JSON:

```
{
  "success": true,
  "data": {}
}
```

GET /api/v1/auth/me

Purpose: Get profile

Request JSON: N/A

Response JSON:

```
{
  "success": true,
  "data": {
    "user": {}
  }
}
```

9. NODE SESSION APIs — JSON

POST /api/v1/sessions

Purpose: Create session

Request JSON:

```
{
  "title": "Toy compliance research"
}
```

Response JSON:

```
{
  "success": true,
  "data": {
    "session": {
      "id": "s123",
      "title": "Toy compliance research"
    }
  }
}
```

GET /api/v1/sessions

Purpose: List sessions

Request JSON: N/A

Response JSON:

```
{
  "success": true,
  "data": []
}
```

GET /api/v1/sessions/:id

Purpose: Session detail

Request JSON: N/A

Response JSON:

```
{
  "success": true,
  "data": {
    "session": {
      "id": "s123",
      "title": "Toy compliance research"
    }
  }
}
```

GET /api/v1/sessions/:id/messages

Purpose: Session messages

Request JSON: N/A

Response JSON:

```
{
  "success": true,
  "data": {
    "messages": []
  }
}
```

10. NODE DATA APIs — JSON

GET /api/v1/standards

Purpose: List/search standards

Parameters: search, filter, sort, pagination

Request JSON: N/A

Response JSON:

```
{
  "success": true,
  "data": []
}
```

GET /api/v1/standards/:id

Purpose: Standard detail

Parameters: Path: id

Request JSON: N/A

Response JSON:

```
{
  "success": true,
  "data": {}
}
```

GET /api/v1/qcos

Purpose: List QCOs

Parameters: standardId

Request JSON: N/A

Response JSON:

```
{
  "success": true,
  "data": []
}
```

GET /api/v1/qcos/:id

Purpose: QCO detail

Parameters: Path: id

Request JSON: N/A

Response JSON:

```
{
  "success": true,
  "data": {}
}
```

GET /api/v1/labs

Purpose: Find labs

Parameters: location, standardId

Request JSON: N/A

Response JSON:

```
{
  "success": true,
  "data": []
}
```

GET /api/v1/labs/:id

Purpose: Lab detail

Parameters: Path: id

Request JSON: N/A

Response JSON:

```
{
  "success": true,
  "data": {}
}
```

GET /api/v1/reports

Purpose: List reports

Request JSON: N/A

Response JSON:

```
{
  "success": true,
  "data": []
}
```

GET /api/v1/reports/:id

Purpose: Report detail

Parameters: Path: id

Request JSON: N/A

Response JSON:

```
{
  "success": true,
  "data": {}
}
```

GET /api/v1/resources

Purpose: List resources

Request JSON: N/A

Response JSON:

```
{
  "success": true,
  "data": []
}
```

GET /api/v1/resources/:id

Purpose: Resource detail

Parameters: Path: id

Request JSON: N/A

Response JSON:

```
{
  "success": true,
  "data": {}
}
```

GET /api/v1/compliance/journeys

Purpose: List compliance journeys

Request JSON: N/A

Response JSON:

```
{
  "success": true,
  "data": []
}
```

POST /api/v1/comparison

Purpose: Compare entities

Request JSON:

```
{
  "entityType": "standard",
  "entityIds": [
    "id1",
    "id2"
  ]
}
```

Response JSON:

```
{
  "success": true,
```

```
{
  "data": {
    "differences": []
  }
}
```

GET /api/v1/saved

Purpose: List saved items

Request JSON: N/A

Response JSON:

```
{
  "success": true,
  "data": []
}
```

POST /api/v1/saved

Purpose: Save item

Request JSON:

```
{
  "itemId": "item123"
}
```

Response JSON:

```
{
  "success": true,
  "data": {
    "success": true
  }
}
```

DELETE /api/v1/saved/:id

Purpose: Remove saved item

Parameters: Path: id

Request JSON: N/A

Response JSON:

```
{
  "success": true,
  "data": {
    "success": true
  }
}
```

11. CHAT CITATION JSON

```
{
  "id": "c123",
  "title": "IS 1293:2019",
  "sourceType": "standard",
  "documentId": "IS-1293-2019",
  "standardNumber": "IS 1293:2019",
  "section": "General requirements",
  "page": 1,
  "url": "https://bis.gov.in/...",
  "verified": true
}
```

Citations are part of the /chat response. No dedicated public evidence endpoint is defined.

12. COMMON JSON RESPONSE FORMAT

Success:

```
{
  "success": true,
  "data": {},
  "meta": {
    "page": 1,
    "total": 1
  },
  "error": null
}
```

Empty list:

```
{
  "success": true,
  "data": []
}
```

Error:

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "RESOURCE_NOT_FOUND",
    "message": "Not found"
  }
}
```

13. FINAL CONNECTION MAP

```
NORMAL API:
React
↓ HTTP/JSON
Node REST
↓
MongoDB
↓
Node JSON
↓
React

AI API:
React
↓ POST /api/v1/chat + JSON
Node REST
↓ POST /chat + JSON
FastAPI AI
↓
RAG → Pinecone → LLM → citations
↓ JSON
Node
↓
React
```

14. FINAL RULE — WHAT TO BUILD

Build all 24 public /api/v1 APIs in Node/Express.

Build FastAPI's internal POST /chat as the AI service API.

Do not make separate public APIs for intent, RAG, Pinecone, LLM or evidence; keep them as internal AI processing under /chat.

For every AI question, the complete AI result is returned from FastAPI to Node as JSON, then Node returns it to the frontend.

Judge-ready: "Node is our public REST gateway. FastAPI is our private AI service. The frontend calls POST /api/v1/chat on Node; Node internally calls POST /chat on FastAPI using HTTP and JSON. FastAPI performs the complete AI processing and returns answer, intent, status and citations. Normal non-AI data remains on Node REST APIs."